المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵉⵏⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

ENSA - BM

École Nationale des Sciences Appliquées

RAPPORT DE TRAVAUX PRATIQUES

Git & Node.js

Gestion de versions, branches, fusion, rebase et collaboration

Étudiant	Fouad BANAN
Module	Génie Logiciel
Filière	Transformation Digitale Industrielle
Encadrant	Pr. BE.ELBAGHAZAOUI
Établissement	ENSA-BM
Année universitaire	2025 / 2026

1. Introduction

Ce rapport présente le compte-rendu du travail pratique portant sur l'utilisation de Git pour la gestion de versions d'un projet Node.js. Git est un système de contrôle de version distribué, devenu un outil incontournable dans le développement logiciel moderne et dans les pratiques DevOps liées à la transformation digitale.

Ce TP couvre l'ensemble du cycle de vie d'un projet : de son initialisation jusqu'à la collaboration via GitHub, en passant par la gestion des branches, la résolution de conflits, le rebase et le cherry-pick.

Objectifs du TP

- Initialiser un projet Node.js avec Git
- Maîtriser la création et la gestion de branches
- Réaliser des fusions (merge) entre branches
- Résoudre des conflits de fusion
- Utiliser le rebase pour une histoire Git linéaire
- Appliquer le cherry-pick pour sélectionner des commits précis
- Collaborer via GitHub (remote, push, pull)

2. Environnement et Outils

Système d'exploitation	Linux / Windows 10+
Git	Version 2.40+
Node.js	Version 18 LTS (ou supérieure)
npm	Inclus avec Node.js
Éditeur de code	Visual Studio Code
Plateforme distante	GitHub

3. Étape 1 : Initialisation du Projet Node.js

La première étape consiste à créer la structure de base du projet, à initialiser le dépôt Git et à effectuer un premier commit.

3.1 Création du dossier et initialisation

```
$ mkdir mon-projet-node
$ cd mon-projet-node
$ git init
$ npm init -y
```

La commande git init crée un dépôt Git local (dossier caché .git/). La commande npm init -y génère automatiquement un fichier package.json avec les valeurs par défaut.

3.2 Fichiers de base : README.md et .gitignore

```
$ echo "# Mon Projet Node" > README.md
$ npx gitignore node
# ou manuellement :
$ echo "node_modules/" >> .gitignore
```

Le fichier .gitignore est essentiel pour exclure le dossier node_modules/ du suivi Git, ce dossier pouvant contenir des centaines de fichiers inutiles à versionner.

3.3 Création d'index.js et premier commit

```
$ echo "console.log('Hello Node');" > index.js
$ git add README.md .gitignore index.js package.json
$ git commit -m "Initialisation du projet Node (README, .gitignore, index.js)"
```

Le script de démarrage peut être ajouté dans package.json pour faciliter l'exécution :

```
// package.json
"scripts": {
  "start": "node index.js"
}

$ npm run start
// Output: Hello Node
```

4. Étape 2 : Gestion des Branches

Les branches permettent de développer des fonctionnalités en parallèle sans impacter la branche principale main. Une branche dev est créée pour ajouter une première fonctionnalité.

4.1 Création de la branche dev

```
$ git branch dev
$ git checkout dev
```

Ces deux commandes peuvent être combinées en une seule : git checkout -b dev

4.2 Ajout de la fonction addition dans index.js

```
// index.js (branche dev)
function addition(a, b) {
  return a + b;
}

if (require.main === module) {
  console.log("Résultat:", addition(5, 3));
}

module.exports = { addition };
```

```
$ git add index.js
$ git commit -m "Ajout de la fonction addition dans index.js"
```

5. Étape 3 : Fusion (Merge)

Une fois la fonctionnalité développée sur la branche dev, elle est fusionnée dans la branche principale main via un merge.

```
$ git checkout main
$ git merge dev
```

Cette fusion est dite fast-forward car la branche main n'a pas évolué depuis la création de dev. Git déplace simplement le pointeur main vers le dernier commit de dev, sans créer de commit de fusion supplémentaire.

6. Étape 4 : Résolution de Conflits

Les conflits apparaissent lorsque deux branches modifient le même fichier aux mêmes lignes. Cette étape simule et résout un tel conflit.

6.1 Création de la branche feature

```
$ git checkout -b feature
```

La branche feature ajoute une fonction soustraction dans index.js :

```
// index.js (branche feature)
function addition(a, b) { return a + b; }

function soustraction(a, b) {
  return a - b;
}

if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}

module.exports = { addition, soustraction };

$ git add index.js
$ git commit -m "Ajout de la fonction soustraction"
```

6.2 Modification sur main (commentaires)

```
$ git checkout main

// index.js (branche main)
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}

$ git add index.js
$ git commit -m "Doc: améliorer les commentaires de addition"
```

6.3 Tentative de fusion et conflit

```
$ git merge feature

Auto-merging index.js
CONFLICT (content): Merge conflict in index.js
Automatic merge failed; fix conflicts and then commit the result.
```

Git signale un conflit car les deux branches ont modifié les mêmes lignes du fichier index.js. Git insère des marqueurs de conflit dans le fichier :

```
<<<<<<< HEAD
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
```

```
}
=======
function addition(a, b) {
  return a + b;
}
function soustraction(a, b) {
  return a - b;
}
>>>>>>> feature
```

6.4 Résolution du conflit

La résolution consiste à combiner manuellement les deux versions pour conserver les commentaires de main ET la fonction soustraction de feature :

```
// index.js (version résolue)
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}

function soustraction(a, b) {
  return a - b;
}

if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}

module.exports = { addition, soustraction };

$ git add index.js
$ git commit   # Git propose un message de merge par défaut
```

7. Étape 5 : Rebase

Le rebase est une alternative au merge pour intégrer des modifications. Il rejoue les commits d'une branche au-dessus d'une autre, produisant un historique Git plus linéaire et lisible.

7.1 Création de la branche bugfix

```
$ git checkout -b bugfix
```

La branche bugfix rend la fonction addition plus robuste en gérant les valeurs NaN :

```
// index.js (bugfix) - rendre addition plus robuste
function addition(a, b) {
  const x = Number(a), y = Number(b);
  if (Number.isNaN(x) || Number.isNaN(y)) return 0;
  return x + y;
}

function soustraction(a, b) { return a - b; }

if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}

module.exports = { addition, soustraction };

$ git add index.js
$ git commit -m "Bugfix: addition robuste (gère les NaN)"
```

7.2 Rebase de main sur bugfix

```
$ git checkout main
$ git rebase bugfix
```

Contrairement au merge, le rebase ne crée pas de commit de fusion. Il réapplique les commits de main au-dessus de bugfix, donnant l'impression que le développement s'est fait de manière séquentielle. Cela simplifie la lecture de l'historique avec git log.

8. Étape 6 : Cherry-pick

Le cherry-pick permet d'appliquer un commit spécifique d'une branche sur une autre, sans fusionner l'intégralité de la branche source.

8.1 Consultation de l'historique

```
$ git log --oneline

3f5c2d3 Bugfix: addition robuste (gère les NaN)
1a2b3c4 Ajout de la fonction soustraction
9d8e7f6 Doc: améliorer les commentaires de addition
```

8.2 Application du commit sur la branche feature

```
$ git checkout feature
$ git cherry-pick 3f5c2d3

# En cas de conflit :
$ git add index.js
$ git cherry-pick --continue
```

Le cherry-pick est particulièrement utile pour appliquer un correctif critique (hotfix) sur plusieurs branches sans attendre une fusion complète.

9. Étape 7 : Collaboration avec GitHub

Cette étape finalise le workflow en associant le dépôt local à un dépôt distant sur GitHub, permettant la collaboration multi-développeurs.

9.1 Configuration de l'identité Git

```
$ git config --global user.name "Fouad BANAN"
$ git config --global user.email "fouad.banan@example.com"
```

9.2 Liaison avec le dépôt GitHub

```
# Création du dépôt vide sur github.com, puis :
$ git remote add origin git@github.com:VotreUser/mon-projet-node.git
$ git branch -M main
$ git push -u origin main
```

9.3 Workflow de collaboration

```
# Récupérer les derniers changements de l'équipe
$ git pull origin main

# Pousser ses propres modifications
$ git push origin main
```

Bonnes pratiques pour la collaboration en équipe :

- Utiliser des branches par fonctionnalité avec la convention feature/nom-feature
- Soumettre des Pull Requests pour la revue de code avant fusion dans main
- Activer la protection de branche sur main pour éviter les pushs directs
- Écrire des messages de commit descriptifs et cohérents
- Effectuer un git pull avant tout développement pour synchroniser avec le remote

10. Synthèse des Résultats

Étape	Résultat
Initialisation Git + Node.js	**Réussi**
Création branche dev	**Réussi**
Fusion (merge) dev -> main	**Réussi**
Résolution de conflit (feature)	**Réussi**
Rebase (bugfix -> main)	**Réussi**
Cherry-pick commit bugfix	**Réussi**
Push vers GitHub	**Fonctionnel**

11. Conclusion

Ce travail pratique a permis de maîtriser les fonctionnalités essentielles de Git dans un contexte de développement Node.js. La gestion des branches, des fusions, des conflits, du rebase et du cherry-pick constituent des compétences fondamentales pour tout développeur travaillant en équipe.

La maîtrise de Git est directement liée aux pratiques DevOps et à la transformation digitale industrielle : versionner le code de manière rigoureuse, collaborer efficacement via des plateformes comme GitHub, et maintenir un historique propre et lisible sont des prérequis indispensables dans tout projet d'industrialisation logicielle.

Les concepts abordés dans ce TP s'inscrivent dans une démarche d'intégration et de déploiement continus (CI/CD), où la qualité du flux Git détermine directement la fiabilité des pipelines de livraison.

Fouad BANAN | Transformation Digitale Industrielle | ENSA-BM